

ROS2 카메라 이미지 실습

카메라 이미지 토픽 퍼블리시 & 스냅샷 서비스

cchyun@gmail.com

Part 1. 시스템 구조 & cv_bridge

- 이미지를 생성하여 보내는 노드 + 이를 받아 처리하는 노드로 구성
 - 노드 1: `image_publisher`
 - 역할: 웹캠 또는 이미지 파일을 읽어 ROS2 토픽으로 전송
 - 발행(Publish) 토픽: `/image_raw` (`sensor_msgs/msg/Image`)
 - 노드 2: `image_processor`
 - 역할: 이미지를 수신하여 화면에 표시하고, 요청 시 이미지를 저장
 - 구독(Subscribe) 토픽: `/image_raw`
 - 서비스 서버: `/capture_snapshot` (`std_srvs/srv/Trigger`)

- ROS2와 OpenCV는 이미지 데이터 형식이 달라서, 중간에 "번역기"가 필요
 - OpenCV
 - 이미지 데이터를 NumPy 배열(ndarray) 형태로 다룸
 - ROS2
 - 이미지 전송을 위해 `sensor_msgs/msg/Image` 형식을 사용
 - `cv_bridge` : 두 형식 사이를 변환해주는 핵심 라이브러리
 - `cv_bridge.cv2_to_imgmsg()` : OpenCV 이미지 → ROS2 메시지
 - `cv_bridge.imgmsg_to_cv2()` : ROS2 메시지 → OpenCV 이미지

- 퍼블리시 노드 (image_publisher)

```
웹캠(OpenCV ndarray) → cv2_to_imgmsg() → /image_raw 토픽 발행
```

- 구독 노드 (image_processor)

```
/image_raw 토픽 수신 → imgmsg_to_cv2() → OpenCV 처리(imshow, imwrite 등)
```

- 핵심 포인트

- ROS2 노드끼리 통신할 때는 반드시 `sensor_msgs/msg/Image` 형식 사용
- OpenCV로 이미지를 보여주거나 저장할 때는 반드시 NumPy 배열로 변환
- 인코딩을 `"bgr8"` 으로 통일 → 색상 이상 방지

Part 2. 단계별 구현

- 터미널에서 작업 공간의 `src` 폴더로 이동하여 패키지를 생성

```
ros2 pkg create --build-type ament_python camera_pkg \  
--dependencies rclpy sensor_msgs std_srvs cv_bridge
```

- 의존성 설명

- `rclpy` : ROS2 Python 클라이언트 라이브러리
- `sensor_msgs` : 이미지 메시지 타입 (`sensor_msgs/msg/Image`)
- `std_srvs` : 표준 서비스 타입 (`std_srvs/srv/Trigger`)
- `cv_bridge` : OpenCV ↔ ROS2 이미지 변환 라이브러리

1단계 — 이미지 퍼블리시 노드 구현

- 웹캠 영상을 읽어 초당 10번(10Hz) 주기로 토픽을 발행
 - 파일명: `camera_pkg/image_publisher.py`
 - 핵심 구조
 - 퍼블리셔 생성: 타입 `Image`, 토픽명 `'image_raw'`, 큐 크기 `10`
 - 타이머 설정: `0.1` 초마다 `timer_callback` 실행 (10Hz)
 - 웹캠 연결: `cv2.VideoCapture(0)` (0번 카메라)
 - `cv2_to_imgmsg(frame, encoding="bgr8")` 로 변환 후 퍼블리시

이미지 퍼블리시 노드 전체 코드

```
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import Image
import cv2
from cv_bridge import CvBridge

class ImagePublisher(Node):
    def __init__(self):
        super().__init__('image_publisher')
        # 1. 퍼블리셔 생성: 타입은 Image, 토픽명은 'image_raw', 큐 크기는 10
        self.publisher_ = self.create_publisher(Image, 'image_raw', 10)
        # 2. 타이머 설정: 0.1초마다 timer_callback 실행 (10Hz)
        self.timer = self.create_timer(0.1, self.timer_callback)
        # 3. 웹캠 연결 (0번 카메라)
        self.cap = cv2.VideoCapture(0)
        self.bridge = CvBridge()

    def timer_callback(self):
        ret, frame = self.cap.read()
        if ret:
            # OpenCV 이미지를 ROS2 이미지 메시지로 변환하여 퍼블리시
            img_msg = self.bridge.cv2_to_imgmsg(frame, encoding="bgr8")
            self.publisher_.publish(img_msg)
            self.get_logger().info('이미지 발행 중...')

def main(args=None):
    rclpy.init(args=args)
    node = ImagePublisher()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    node.destroy_node()
    rclpy.shutdown()
```

2단계 & 3단계 — 이미지 구독 & 캡처 서비스 노드

- 이미지를 받아 화면에 보여주면서, 서비스 요청이 오면 현재 화면을 파일로 저장

- 파일명: `camera_pkg/image_processor.py`

- **핵심 구조**

- 서브스크라이버: `'image_raw'` 토픽 구독 → `image_callback`
- 서비스 서버: `'capture_snapshot'` 서비스 제공 → `capture_callback`
- `imgmsg_to_cv2(msg, desired_encoding="bgr8")` 로 변환
- `cv2.imshow()` 로 화면 표시, `cv2.imwrite()` 로 파일 저장
- `cv2.waitKey(1)` : 화면 갱신을 위해 필수

이미지 구독 & 캡처 노드 전체 코드

```
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import Image
from std_srvs.srv import Trigger
import cv2
from cv_bridge import CvBridge

class ImageProcessor(Node):
    def __init__(self):
        super().__init__('image_processor')
        self.bridge = CvBridge()
        self.current_frame = None
        # 1. 서브스크라이버 생성: 'image_raw' 토픽 구독
        self.subscription = self.create_subscription(
            Image, 'image_raw', self.image_callback, 10)
        # 2. 서비스 서버 생성: 'capture_snapshot' 서비스 제공
        self.srv = self.create_service(
            Trigger, 'capture_snapshot', self.capture_callback)

    def image_callback(self, msg):
        # ROS2 메시지를 OpenCV 이미지로 변환
        self.current_frame = self.bridge.imgmsg_to_cv2(msg, desired_encoding="bgr8")
        cv2.imshow("Camera View", self.current_frame)
        cv2.waitKey(1) # 화면 갱신을 위해 필수

    def capture_callback(self, request, response):
        if self.current_frame is not None:
            cv2.imwrite('snapshot.jpg', self.current_frame)
            response.success = True
            response.message = "스냅샷이 snapshot.jpg로 저장되었습니다!"
        else:
            response.success = False
            response.message = "이미지가 아직 수신되지 않았습니다."
        return response

def main(args=None):
    rclpy.init(args=args)
    node = ImageProcessor()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()
```

4단계 — 빌드 및 실행 확인

- `setup.py` 등록 : `console_scripts` 에 두 노드를 추가

```
entry_points={
    'console_scripts': [
        'img_pub = camera_pkg.image_publisher:main',
        'img_proc = camera_pkg.image_processor:main',
    ],
},
```

- 빌드

```
colcon build
source install/setup.bash
```

4단계 — 실행 및 테스트

- 각각 다른 터미널에서 실행

```
# 터미널 1
ros2 run camera_pkg img_pub

# 터미널 2
ros2 run camera_pkg img_proc
```

- 시각화 확인 : `rqt_image_view`로 토픽이 잘 나오는지 확인

```
ros2 run rqt_image_view rqt_image_view
```

- 스냅샷 서비스 호출 테스트

```
ros2 service call /capture_snapshot std_srvs/srv/Trigger {}
```

Part 3. 예외 처리 & 심화 미션

자주 발생하는 오류와 해결법

- ImportError: No module named 'cv_bridge'

- 원인: `cv_bridge` 패키지가 설치되지 않았거나 환경 변수가 설정되지 않음
- 해결:

```
sudo apt install ros-humble-cv-bridge
source /opt/ros/humble/setup.bash
```

- 이미지 인코딩 불일치 (Encoding Mismatch)

- 원인: `bgr8` (OpenCV 기본)과 `rgb8` 을 혼동하여 색상이 이상하게 나옴
- 해결: `cv2_to_imgmsg` 와 `imgmsg_to_cv2` 호출 시 인코딩을 `"bgr8"` 으로 통일

- 화면이 멈춤 (GUI Freeze)

- 원인: `cv2.imshow()` 후에 `cv2.waitKey(1)` 이 없으면 GUI 창이 응답하지 않음
- 해결: `image_callback` 안에 반드시 `cv2.waitKey(1)` 추가



• 미션 1: YOLO 결과 포함

- 이미지 위에 `cv2.rectangle` 로 박스를 그려서 퍼블리시해 보세요

```
cv2.rectangle(frame, (x1, y1), (x2, y2), (0, 255, 0), 2)
img_msg = self.bridge.cv2_to_imgmsg(frame, encoding="bgr8")
self.publisher_.publish(img_msg)
```

• 미션 2: Canny 엣지 재발행

- 구독 노드에서 `cv2.Canny()` 처리한 이미지를 `/image_edge` 새 토픽으로 퍼블리시

```
edge = cv2.Canny(self.current_frame, 100, 200)
self.edge_publisher.publish(self.bridge.cv2_to_imgmsg(edge, "mono8"))
```

• 미션 3: 동시 시각화

- `rqt_image_view` 를 두 개 띄워 원본 영상과 엣지 영상을 동시에 확인